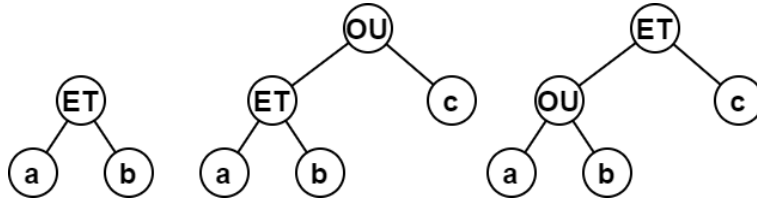


## Les arbres – exercices

### 1. Les parcours en profondeur.

1) **Donner** les parcours préfixe, infixe et postfixe des arbres suivants :



Pré : ET a b | in : a ET b | post : a b ET

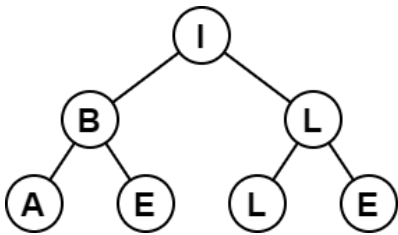
Pré : OU ET a b c | in : a ET b OU c | post : a b ET c OU

Pré : ET OU a b c | in : a OU b ET c | post : a b OU c ET

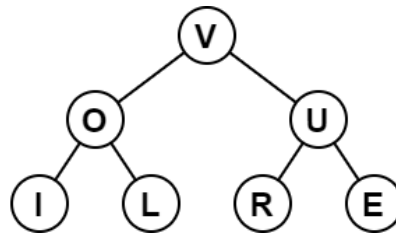
→ Donne une idée de la priorité des opérateurs selon la profondeur du nœud.

Lien avec les expressions infixe/postfixe

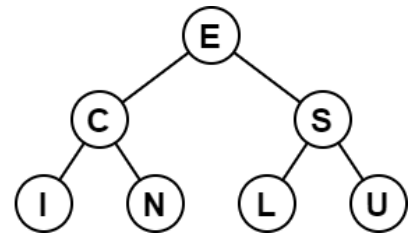
2) Les arbres suivants mémorisent un mot. Indiquer quel parcours réaliser pour les retrouver.



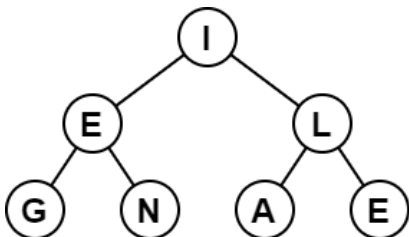
Abeille : infixe



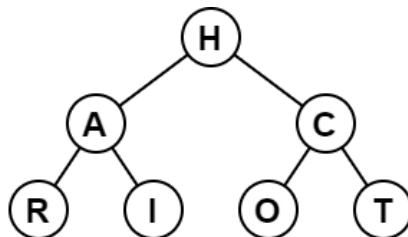
Voilure : préfixe



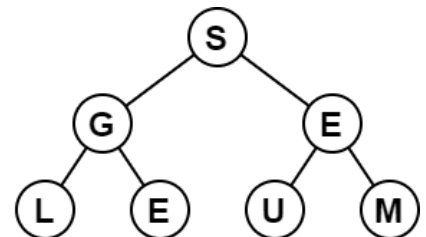
Incluse : postfixe



Géniale : infixe



Haricot : préfixe

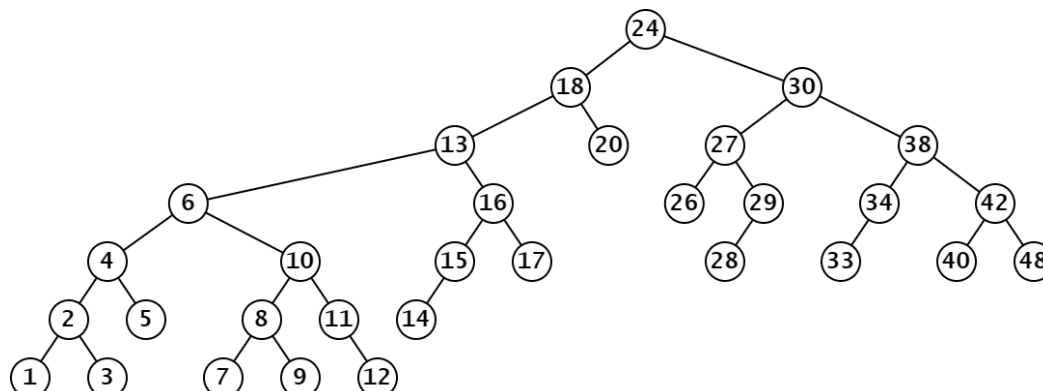


Légumes : postfixe

## 2. Les arbres binaires de recherche

### 2.1. Retrouver une valeur

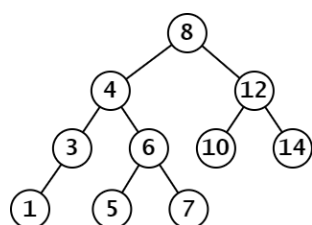
Soit l'arbre suivant. **indiquer** quel chemin parcourir pour parvenir aux valeurs demandées ci-dessous. Vous pouvez dessiner/mettre en évidence le chemin, ou donner la liste des nœuds parcourus.



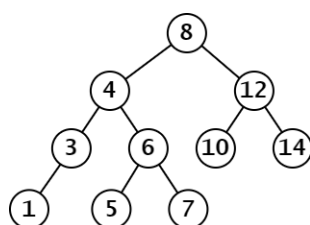
Valeurs à rechercher :  $24 \rightarrow 18 \rightarrow 20$ ,  $24 \rightarrow 18 \rightarrow 13 \rightarrow 6 \rightarrow 4 \rightarrow 5$ ,  $24 \rightarrow 18 \rightarrow 13 \rightarrow 6 \rightarrow 10 \rightarrow 11 \rightarrow 12$ ,  
 $24 \rightarrow 30 \rightarrow 27 \rightarrow 29 \rightarrow 28$ ,  $24 \rightarrow 30 \rightarrow 38 \rightarrow 42 \rightarrow 48$ ,  $24 \rightarrow 18 \rightarrow 13 \rightarrow 6 \rightarrow 4 \rightarrow 2 \rightarrow 1$ ,  
 $24 \rightarrow 30 \rightarrow 27$ ,  $24 \rightarrow 18 \rightarrow 13$ .

### 2.2. Insérer une valeur

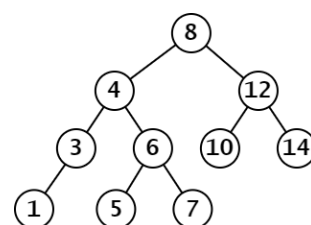
Dans les arbres suivants, **insérer** dans l'ordre les valeurs demandées :



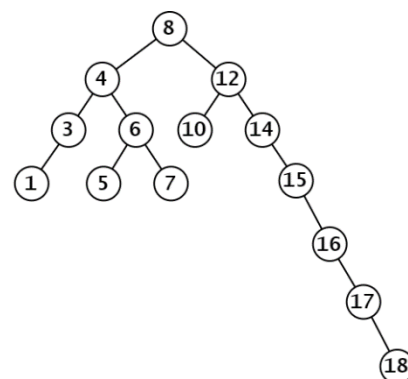
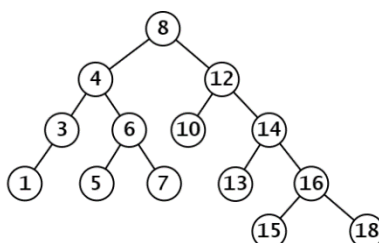
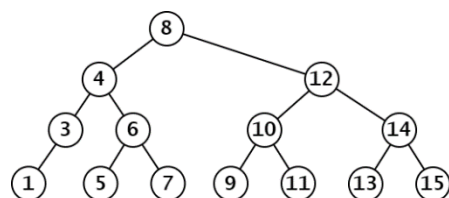
11, 9, 13, 15.



16, 18, 15, 13.



15, 16, 17, 18

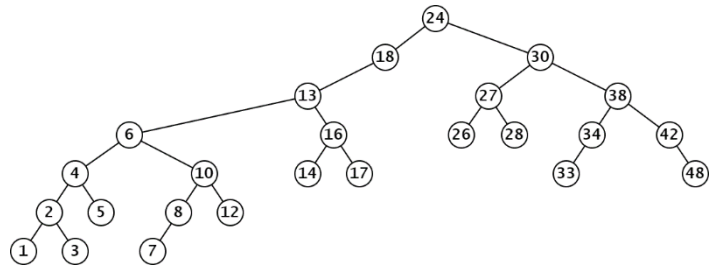


## 2.3. Supprimer une valeur

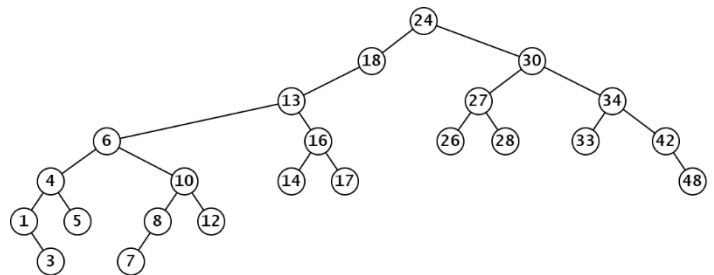
Dans l'arbre présenté au point 2.1, **supprimer** les nœuds suivants. Pour chaque suppression, vous redessinerez la partie de l'arbre concerné par la suppression.

Nœuds à supprimer : 20, 40, 9, 29, 15, 11, 2, 38, 13

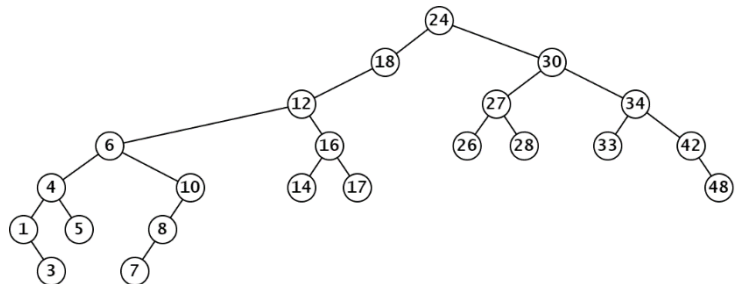
Suppression des feuilles 20, 40, 9 et suppression des nœuds internes 29, 15, 11 :



Suppression nœuds 2 et 38 :

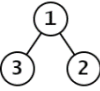
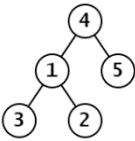
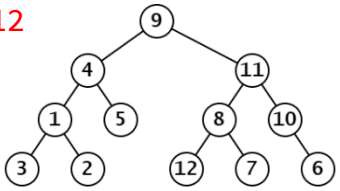
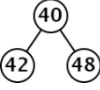
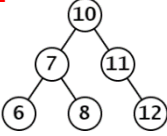
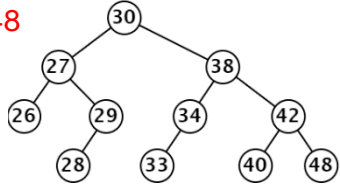


Suppression du nœud 13 :



## 2.4. Retrouver une valeur maximum

- 1) Dans les arbres suivants, **retrouver** le nœud ayant la valeur maximum. Vous prêterez attention à la manière dont vous parcourez l'arbre : mettre en évidence le parcours utilisé et le raisonnement appliqué. **Penser à la récursivité.**

|                             |                                                                                         |                                                                                         |                                                                                           |
|-----------------------------|-----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| Arbres binaires quelconque  | 3<br>  | 5<br>  | 12<br> |
| Arbre binaires de recherche | 48<br> | 12<br> | 48<br> |

- 2) A partir des résultats obtenus à la question 1), **écrire** un algorithme de recherche de maximum dans un Arbre Binaire de Recherche et dans un Arbre Binaire quelconque.

Pour un ABR :

```
Fonction maximum(nœud)
    Si nœud est une feuille alors
        renvoyer l'étiquette
    Sinon
        Si fils droit de nœud existe alors
            renvoyer maximum(nœud.fils_droit)
```

Pour un arbre binaire quelconque :

```
Fonction maximum(nœud)
    Si nœud est une feuille alors
        renvoyer l'étiquette
    Si fils gauche de nœud existe alors
        max_gauche = maximum(nœud.fils_gauche)
    sinon
        max_gauche = None
    Si fils droit de nœud existe alors
        max_droite = maximum(nœud.fils_droite)
    sinon
        max_droite = None
    si max_gauche > max_droite alors // on sous-entend que None peut être comparable
        renvoyer max_gauche
    sinon
        renvoyer max_droite
```